

✓ Predicting Stroke Risk

Individual Project:

Student Name: Amna

Student ID: GH1039590

Subject code: B104

Submission Date: 18/09/2026

Subject name: Artificial Intelligence & Machine Learning(SS0626)

Course: Data Science, AI and Digital Business

Institution: Gisma University of Applied Sciences

Project Topic: Predicting Stroke Risk: An End-to-End Machine Learning Pipeline

Dataset: [Kaggle — Stroke Prediction Dataset](#)

GitHub Repository: [My-Github-Link](#)

```
# Requirements and importing the libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import shap

from sklearn.model_selection import train_test_split, GridSearchCV,
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostin
from sklearn.metrics import (classification_report, confusion_matrix
                             roc_auc_score, roc_curve, precision_re

pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')
```

✓ 1. Problem Statement

Business problem:

Stroke is one of the top two causes of death in the world. A health insurer / primary-care network would like to identify patients who are at high risk of stroke before they become stroke patients to enable their prioritisation for having a preventive checkup (blood-pressure control, lifestyle counselling, review on anticoagulation etc.).

Why it is important to the company:

Early risk flagging will decrease the number of admission to hospital in an emergency, disability after sickness or disability following hospitalisation, enable a small clinical team to attend to people with the highest risk—with limited time on everyone equally, and to improve outcomes.

Data collection:

These data would be obtained from patients' electronic health records (EHRs): Demographics, vital signs (blood glucose, BMI), comorbidities (hypertension, heart disease), and lifestyle information (smoking status, work type). For this project we use a public Kaggle dataset of '**Stroke Prediction**' with fedesoriano, which is also similar to the type of EHR extract (this just has only 5,110 patients and 12 columns).

ML formulation. This is a two-class classification problem:

- **Input X** : Take patient demographic, clinical and lifestyle attributes.
- **Output y** : `stroke` (1 = had a stroke, 0 = did not).

The model to be used should be intended to generate a probability (risk score) and not simply a hard classification, providing clinicians with a guidance on how to classify or prioritise patients by risk range rather than a single score.

```
# Loading the dataset from KaggleHub
```

```
import kagglehub
path = kagglehub.dataset_download("fedesoriano/stroke-prediction-da

import os
file_path = os.path.join(path, "healthcare-dataset-stroke-data.csv"
data = pd.read_csv(file_path)

print(f"Dataset loaded: {data.shape[0]:,} rows, {data.shape[1]} col
data.head()
```

Using Colab cache for faster access to the 'stroke-prediction-datase
Dataset loaded: 5,110 rows, 12 columns

	id	gender	age	hypertension	heart_disease	ever_married	work
0	9046	Male	67.0	0	1	Yes	
1	51676	Female	61.0	0	0	Yes	en
2	31112	Male	80.0	0	1	Yes	
3	60182	Female	49.0	0	0	Yes	

Next steps:

[Generate code with data](#)[New interactive sheet](#)

✓ 2. Data Exploration

Data quality

- **Standard missing value counts:** ~201 missing values in the `bmi` column ($\approx 3.9\%$ of the rows), no other column has missing values.
- **Duplicate / irrelevant column:** `id` A unique identifier of a patient, ID does not include predictive information — ID is dropped before modelling.
- **Rare categories:** `gender` has a single "Other" row; `work_type` has very few "Never_worked" rows. Both are kept (one-hot encoding handles them), but they contribute little statistical power.

Class imbalance

Only ~4.9% of patients in this dataset had a stroke (249 of 5,110). This is highly skewed target and has two impacts:

- **Accuracy is misleading** — It is impossible to decide accuracy, as 1 would predict "no stroke" all the time, with an accuracy of ~95%, but would have no clinical relevance.

Instead we use recall (sensitivity),** F1-score, ROC-AUC and PR-AUC –** and we create the imbalance during training via the `class_weight` argument, which we set to 'balanced', because of the cost difference between missing out on the at-risk patient (false negative) and the unnecessary check-up (false positive).

Sampling / balancing

Because the absolute number of positive cases is small, class weights are used instead of oversampling using SMOTE but they do reduce the number of representative samples for the majority class. Since there are little number of positive cases, we don't work with undersampling because the majority class would be discarded, but don't use oversampling in **SMOTE** technique which would necessitate using many samples.

```
#the overall size of our dataset (rows and columns)
#checking for any missing values to clean later on
def explore_data(df):
    print(f"Shape: {df.shape}")
    print(f"\nMissing values:\n{df.isna().sum()[df.isna().sum() > 0]}")
    print(f"\nTarget distribution:")
    counts = df['stroke'].value_counts()
    for label, count in counts.items():
        print(f"  stroke={label}: {count}, ({count/len(df)*100:.1f}%")
    print(f"\nNumeric summary:\n{df[['age', 'avg_glucose_level', 'bmi']]}")

explore_data(data)
```

Shape: (5110, 12)

Missing values:

bmi 201

dtype: int64

Target distribution:

stroke=0: 4,861 (95.1%)

stroke=1: 249 (4.9%)

Numeric summary:

	age	avg_glucose_level	bmi
count	5110.000000	5110.000000	4909.000000
mean	43.226614	106.147677	28.893237
std	22.612647	45.283560	7.854067
min	0.080000	55.120000	10.300000
25%	25.000000	77.245000	23.500000
50%	45.000000	91.885000	28.100000
75%	61.000000	114.090000	33.100000
max	82.000000	271.740000	97.600000

```
# Initializing subplot grid for comparative exploratory data visual
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('Stroke Dataset – Key Distributions', fontweight='bold')

#Visualizing target variable class distribution to access imbalance

data['stroke'].value_counts().plot(kind='bar', ax=axes[0], color=['
axes[0].set_title('Class balance (0=no stroke, 1=stroke)')
axes[0].set_xlabel('stroke')

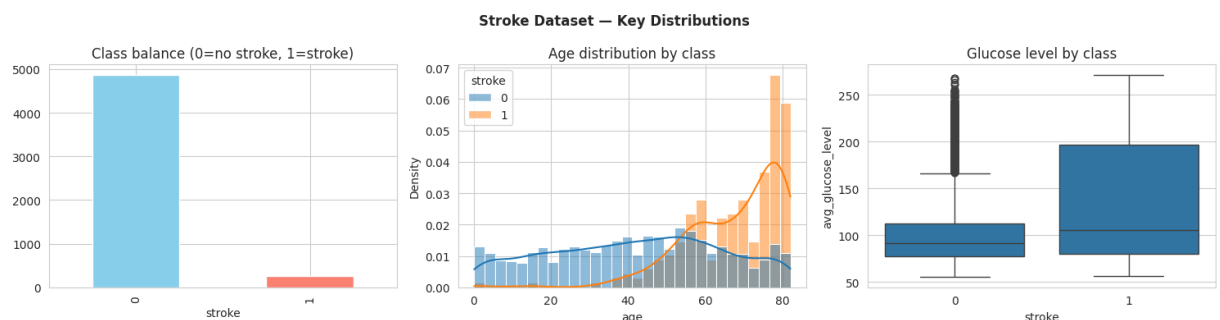
#Examining age distribution segment

sns.histplot(data=data, x='age', hue='stroke', bins=30, ax=axes[1],
axes[1].set_title('Age distribution by class')

#Comparing central tendency & spread of avg glucose level across

sns.boxplot(data=data, x='stroke', y='avg_glucose_level', ax=axes[2]
axes[2].set_title('Glucose level by class')

plt.tight_layout()
plt.show()
```



```
# Bivariate: stroke rate by known clinical risk factors
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
```

```

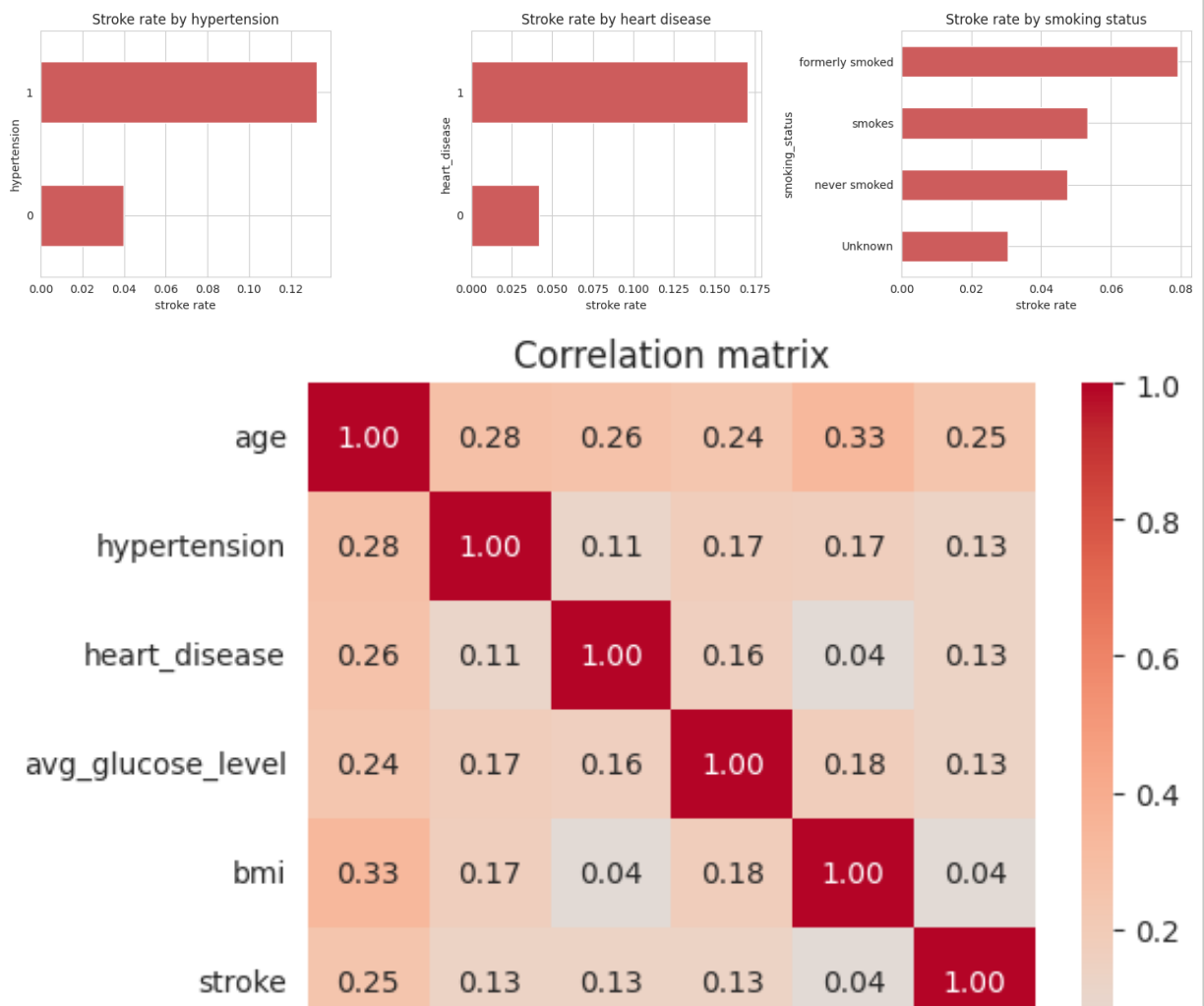
for ax, col, title in zip(
    axes,
    ['hypertension', 'heart_disease', 'smoking_status'],
    ['Stroke rate by hypertension', 'Stroke rate by heart disease',
    ]):

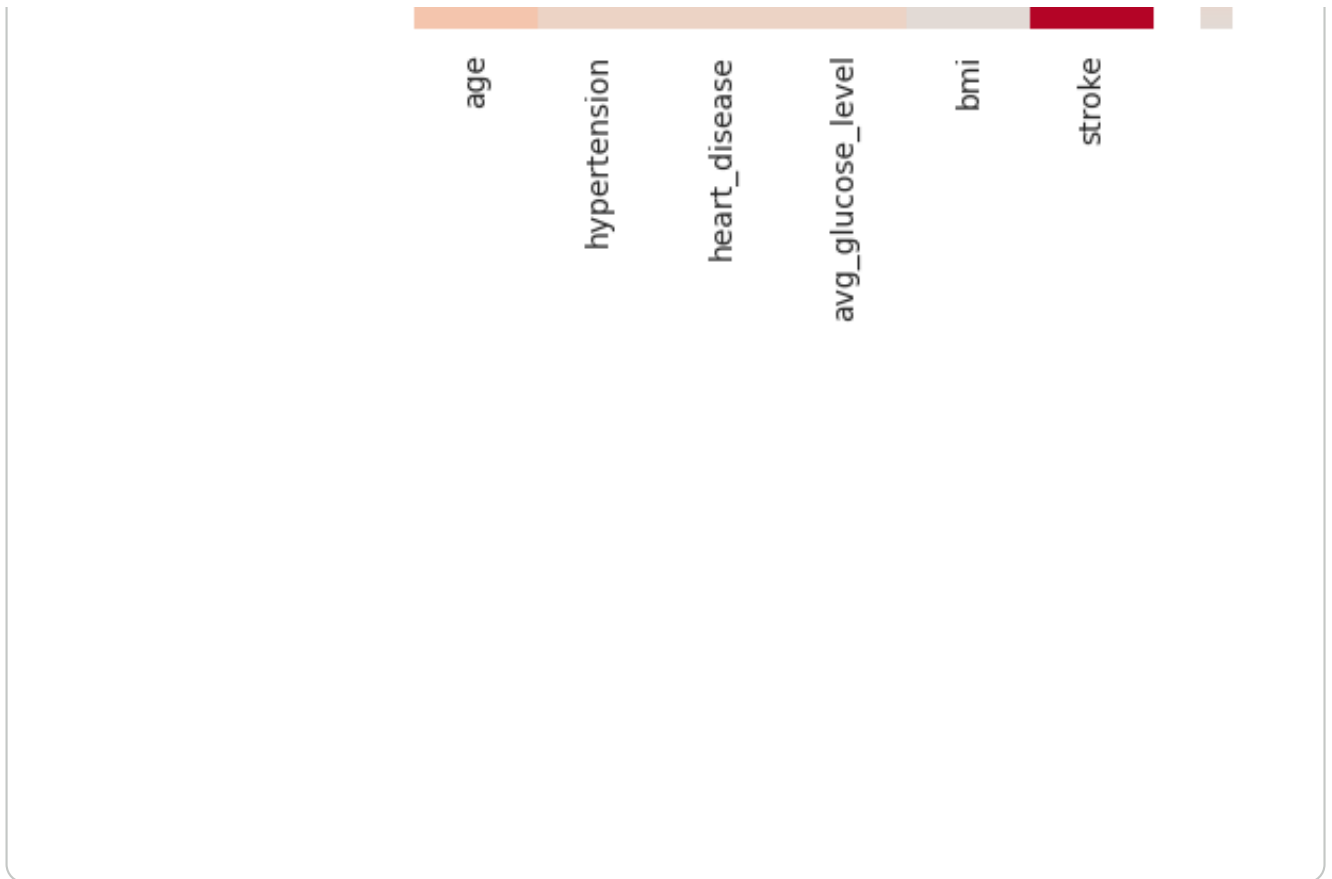
#calculating the mean target rate as per the category & sorting the

    rate = data.groupby(col)['stroke'].mean().sort_values()
    rate.plot(kind='barh', ax=ax, color='indianred')
    ax.set_title(title)
    ax.set_xlabel('stroke rate')
plt.tight_layout()
plt.show()

#Correlation heatmap of numeric & binary fields
corr_cols = ['age', 'hypertension', 'heart_disease', 'avg_glucose_l
plt.figure(figsize=(6, 5))
sns.heatmap(data[corr_cols].corr(), annot=True, fmt='.2f', cmap='cc
plt.title('Correlation matrix')
plt.tight_layout()
plt.show()

```





✓ 3. Data Preprocessing and Feature Engineering

Steps applied (for avoiding leakage, only fit during the training split):

1. **Drop** `id` — which are NOT predictive.
2. **Train/test split** — Data split is trained and tested 80/20 based on the stroke stratified data keeping the same positive rate of ~4.9% for both the trained set and the test set.
3. **New feature** — `age_group`: bucket by `age` into clinically meaningful bands (`child`, `adult`, `middle_age`, `senior`) in addition to `age` which is a continuous variable, to facilitate non-linear age thresholds being picked up by a tree model.
4. **New feature** — `glucose_risk`: `avg_glucose_level > 140` binary type flag indicating whether the glucose value is above the diabetes threshold of 140 (pre-diabetic/diabetic) or not.
5. **Impute** `bmi` — median impute (handling BMI outliers).
6. **Encode categoricals** Categoricals (`gender`, `ever_married`, `work_type`, `Residence_type`, `smoking_status`, `age_group`)— One Hot Encoding.
7. **Scale numerics** — most commonly for logistic regression model baseline, tree numerics are scale-invariant but it won't cause any harm.

All elements are encapsulated in a single scikit-learn `Pipeline` + `ColumnTransformer`, it means the same transformation will automatically happen at the time of inference.

```

RANDOM_STATE = 42
def engineer_features(df):
    df = df.copy()
    df['age_group'] = pd.cut(df['age'], bins=[0, 12, 18, 45, 65, 120],
                             labels=['child', 'teen', 'adult', 'mid', 'old'])

    #creating binary clinical risk indicator for elevated avg glucose
    df['glucose_risk'] = (df['avg_glucose_level'] > 140).astype(int)
    return df

data_fe = engineer_features(data)

X = data_fe.drop(columns=['id', 'stroke'])
y = data_fe['stroke']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
print(f"Train: {X_train.shape[0]:,} rows ({y_train.mean()*100:.1f}% positive)")
print(f"Test: {X_test.shape[0]:,} rows ({y_test.mean()*100:.1f}% positive)")

numeric_features = ['age', 'avg_glucose_level', 'bmi', 'hypertension', 'stroke', 'Residence_type']
categorical_features = ['gender', 'ever_married', 'work_type', 'Residence_type']

numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])

categorical_pipe = Pipeline([
    ('onehot', OneHotEncoder(handle_unknown='ignore')),
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipe, numeric_features),
    ('cat', categorical_pipe, categorical_features),
])

```

```

Train: 4,088 rows (4.9% positive)
Test: 1,022 rows (4.9% positive)

```

✓ 4. Model Training & Hyperparameter Tuning

Comparing **three families of models** which were all configured with

`class_weight='balanced'` when it was parameterizable:

- **Logistic Regression** — linear baseline model that is easily interpretable in terms of risk multipliers (coefficients).
- **Random Forest** — contains non-linear interactions (e.g. age x hypertension).
- **Gradient Boosting** — typically the top-notch presenter for tabular data, but with a lack of interpretability (addressed later with SHAP).

Stratified **5-fold cross validation** is used to tune parameters, and **F1-score**, a combination of precision and recall, is used to do so best suited to this clinically-costly-false-negative scenario than accuracy.

```

#setting-up,K-fold cross-validation to preserve class distribution

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_

#defining model dictionary containing pipelines

models = {
    'logreg': (
        Pipeline([('prep', preprocessor), ('clf', LogisticRegressio
        {'clf__C': [0.01, 0.1, 1, 10]}
    ),
    'random_forest': (
        Pipeline([('prep', preprocessor), ('clf', RandomForestClass
        {'clf__n_estimators': [200, 400], 'clf__max_depth': [4, 8,
    ),
    'gradient_boosting': (
        Pipeline([('prep', preprocessor), ('clf', GradientBoostingC
        {'clf__n_estimators': [100, 200], 'clf__max_depth': [2, 3],
    ),
}
#initializing containers for best estimators
#storing the best results for each model
best_estimators, cv_scores = {}, {}
for name, (pipe, grid) in models.items():
    gs = GridSearchCV(pipe, grid, scoring='f1', cv=cv, n_jobs=-1)
    gs.fit(X_train, y_train)
    best_estimators[name] = gs.best_estimator_
    cv_scores[name] = gs.best_score_

    print(f"{name}: best CV F1 = {gs.best_score_:.3f}, best params

logreg: best CV F1 = 0.222, best params = {'clf__C': 0.01}
random_forest: best CV F1 = 0.257, best params = {'clf__max_depth':
gradient_boosting: best CV F1 = 0.059, best params = {'clf__learning

```

✓ 4.1 Handling imbalance a second way — SMOTE comparison

We re-train the best model family with **SMOTE** (Synthetic Minority Over-sampling) inside the CV loop (we do not use SMOTE on the test set because it would cause synthetic points to be "leaked") and compare CV F1 with the F1 score obtained from the class weighted setting.

```

smote_pipe = ImbPipeline([
    ('prep', preprocessor),
    ('smote', SMOTE(random_state=RANDOM_STATE)),
    ('clf', RandomForestClassifier(random_state=RANDOM_STATE)),
])
smote_grid = {'clf__n_estimators': [200, 400], 'clf__max_depth': [4
gs_smote = GridSearchCV(smote_pipe, smote_grid, scoring='f1', cv=cv
gs_smote.fit(X_train, y_train)

print(f"Random Forest + class_weight  CV F1: {cv_scores['random_for
print(f"Random Forest + SMOTE          CV F1: {gs_smote.best_score_
#whichever scores higher on CV F1 is carried forward as "random_for

if gs_smote.best_score_ > cv_scores['random_forest']:
    best_estimators['random_forest'] = gs_smote.best_estimator_
    cv_scores['random_forest'] = gs_smote.best_score_
    print("-> SMOTE version adopted as the Random Forest candidate.
else:
    print("-> class-weighted version retained as the Random Forest

Random Forest + class_weight  CV F1: 0.257
Random Forest + SMOTE          CV F1: 0.223
-> class-weighted version retained as the Random Forest candidate.

```

✓ 5. Model Assessment on the Held-Out Test Set

All three models tested are reported with a test split that has not been touched by data splitting.

Recall and precision, F1, ROC-AUC and PR-AUC are reported across all three models for the untouched test split. As well as ROC-AUC, PR-AUC (also known as area under the precision-recall curve) is reported, as it is more informative on such an imbalanced data set.

```

#Initializing lists for evaluation results
#evaluating each optimized model on the test
results = []
for name, est in best_estimators.items():
    y_pred = est.predict(X_test)
    y_proba = est.predict_proba(X_test)[:, 1]

    report = classification_report(y_test, y_pred, target_names=['n
    results.append({
        'model': name,

```

```

        'recall (stroke)': report['stroke']['recall'],
        'precision (stroke)': report['stroke']['precision'],
        'f1 (stroke)': report['stroke']['f1-score'],
        'roc_auc': roc_auc_score(y_test, y_proba),
        'pr_auc': average_precision_score(y_test, y_proba),
    })

    print(f"\n=== {name} ===")
    print(classification_report(y_test, y_pred, target_names=['no s

results_df = pd.DataFrame(results).set_index('model').round(3)
print("\nModel comparison on held-out test set:")
results_df

```

```

=== logreg ===

```

	precision	recall	f1-score	support
no stroke	0.99	0.71	0.83	972
stroke	0.12	0.80	0.22	50
accuracy			0.72	1022
macro avg	0.56	0.76	0.52	1022
weighted avg	0.94	0.72	0.80	1022

```

=== random_forest ===

```

	precision	recall	f1-score	support
no stroke	0.98	0.85	0.91	972
stroke	0.20	0.72	0.31	50
accuracy			0.84	1022
macro avg	0.59	0.78	0.61	1022
weighted avg	0.94	0.84	0.88	1022

```

=== gradient_boosting ===

```

	precision	recall	f1-score	support
no stroke	0.95	0.99	0.97	972
stroke	0.44	0.08	0.14	50
accuracy			0.95	1022
macro avg	0.70	0.54	0.55	1022
weighted avg	0.93	0.95	0.93	1022

Model comparison on held-out test set:

	recall (stroke)	precision (stroke)	f1 (stroke)	roc_auc	pr_auc
--	--------------------	-----------------------	----------------	---------	--------



model					
logreg	0.80	0.125	0.216	0.837	0.244
random_forest	0.72	0.198	0.310	0.838	0.236
gradient_boosting	0.08	0.444	0.136	0.814	0.192

Next steps: [Generate code with results_df](#)

[New interactive sheet](#)

Table reading: Gradient Boosting has the best accuracy rates (although spelled out in Section 4 it has the worst recall because it has no possibility for weighting minority class).

This is a real case scenario to provide a certain sense of why you aren't measuring accuracy for this problem. **Logistic Regression** is the one with the highest recall, and the lowest precision (over-flags).

Random Forest gives the best overall balance (highest F1) and is carried forward as the final model.

```
#initializing evaluation subplot grid

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
colors = {'logreg': 'tab:blue', 'random_forest': 'tab:orange', 'gra

#compute & plot ROC & PR curves

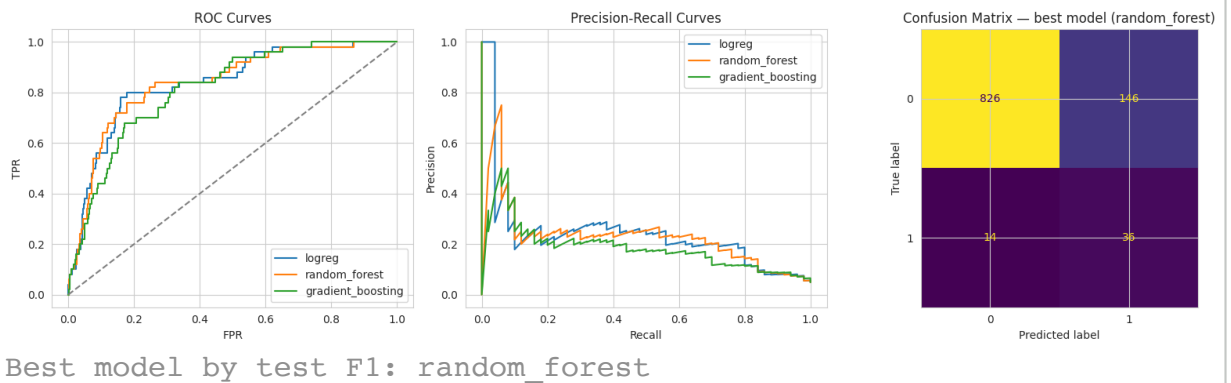
for name, est in best_estimators.items():
    y_proba = est.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    axes[0].plot(fpr, tpr, label=name, color=colors[name])
    prec, rec, _ = precision_recall_curve(y_test, y_proba)
    axes[1].plot(rec, prec, label=name, color=colors[name])

#format ROC curve panel with random baseline

axes[0].plot([0, 1], [0, 1], '--', color='gray')
axes[0].set_title('ROC Curves'); axes[0].set_xlabel('FPR'); axes[0]
axes[1].set_title('Precision-Recall Curves'); axes[1].set_xlabel('R

best_name = results_df['f1 (stroke)'].idxmax()
y_pred_best = best_estimators[best_name].predict(X_test)
ConfusionMatrixDisplay.from_predictions(y_test, y_pred_best, ax=ax=axe
axes[2].set_title(f'Confusion Matrix – best model ({best_name})')
```

```
plt.tight_layout(); plt.show()
print(f"Best model by test F1: {best_name}")
```



✓ 5.1 Explainability - global feature importance and SHAP

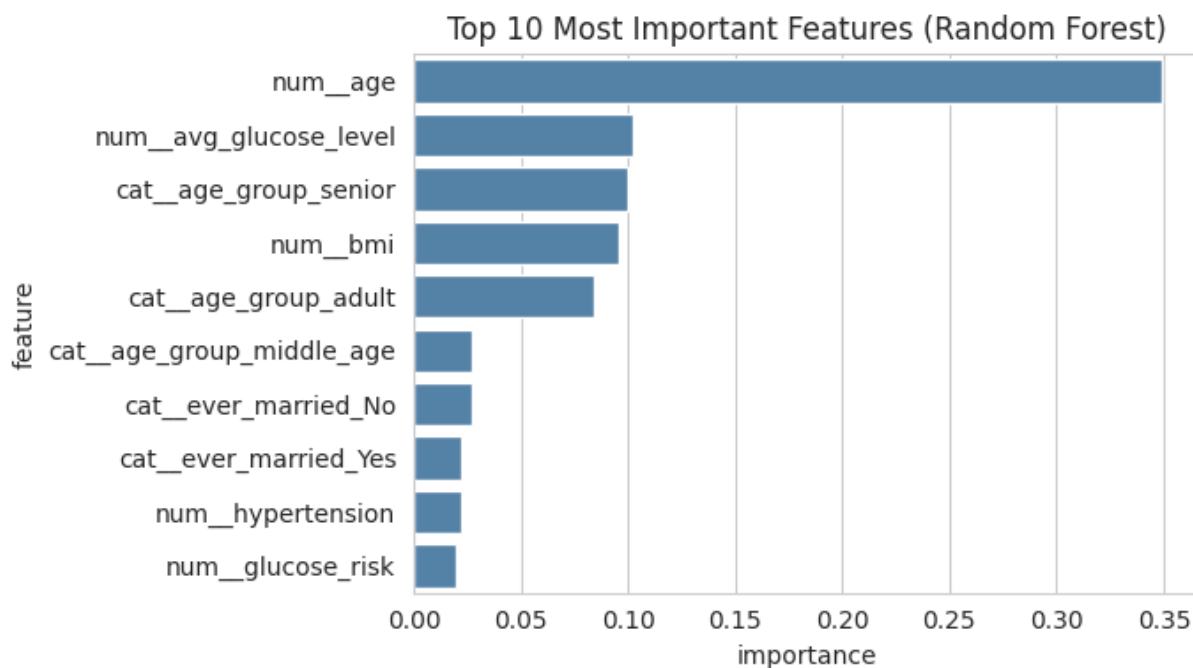
To get a general overview I use the RF's inbuilt feature importances and to get a deeper understanding of how each feature affects each prediction, using **SHAP** values on the best model which provides a model agnostic explanation of the effect each feature has on an individual prediction, such as a flagged case that a clinician would want to trust.

```
#extracting the optimized Random Forest
rf = best_estimators['random_forest']
feature_names = rf.named_steps['prep'].get_feature_names_out()
importances = rf.named_steps['clf'].feature_importances_

imp_df = pd.DataFrame({'feature': feature_names, 'importance': impc

#plotting the top 10 most important features for interpretability
```

```
plt.figure(figsize=(7, 4))
sns.barplot(data=imp_df, x='importance', y='feature', color='steelblue')
plt.title('Top 10 Most Important Features (Random Forest)')
plt.tight_layout(); plt.show()
imp_df
```



	feature	importance	
0	num__age	0.349064	
1	num__avg_glucose_level	0.102366	
25	cat__age_group_senior	0.100275	
2	num__bmi	0.095457	
22	cat__age_group_adult	0.084044	
24	cat__age_group_middle_age	0.027013	
9	cat__ever_married_No	0.026995	
10	cat__ever_married_Yes	0.022505	
3	num__hypertension	0.021882	
5	num__glucose_risk	0.019980	

Next steps: [Generate code with imp_df](#)

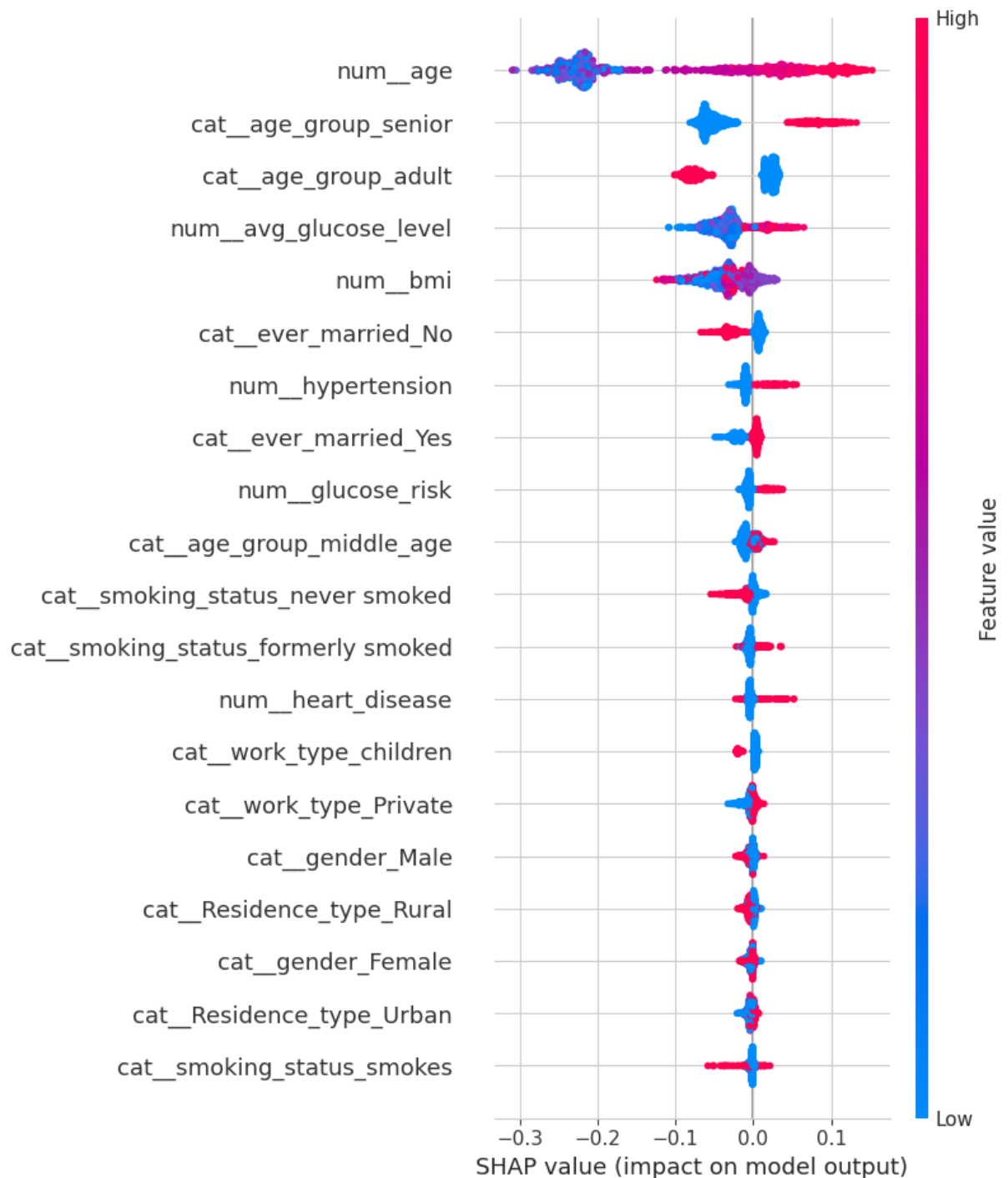
[New interactive sheet](#)

```

best_model = best_estimators[best_name]
X_test_transformed = best_model.named_steps['prep'].transform(X_test)
explainer = shap.TreeExplainer(best_model.named_steps['clf'])
shap_values = explainer.shap_values(X_test_transformed)
sv = shap_values[1] if isinstance(shap_values, list) else shap_valu
if sv.ndim == 3:
    sv = sv[:, :, 1]

```

```
shap.summary_plot(sv, X_test_transformed, feature_names=feature_names)
```



✓ 6. Function Workflow

To demonstrate how a brand new patient's raw data goes through the trained pipeline with trained features to generate a useful "statin or no?"

output, `predict_stroke_risk()` would take the patient data, impute it with the same features used for the training (`age_group`, `glucose_risk`), pass it through the same pipeline of preprocessing and Random Forest, and then return the resulting probability as a three-level "statin or no?" label which the physician would be able to act on.

```
def predict_stroke_risk(patient: dict, model=best_model):
    df_in = pd.DataFrame([patient])
    df_in = engineer_features(df_in)
    proba = model.predict_proba(df_in)[0, 1]
    tier = 'HIGH' if proba >= 0.5 else ('MEDIUM' if proba >= 0.2 el
    return proba, tier

example_patient = {
    'gender': 'Male', 'age': 67, 'hypertension': 0, 'heart_disease'
    'work_type': 'Private', 'Residence_type': 'Urban', 'avg_glucose'
    'smoking_status': 'formerly smoked'
}

proba, tier = predict_stroke_risk(example_patient)
print(f"Patient: {example_patient}")
print(f"\nPredicted stroke probability: {proba:.3f}")
print(f"Risk tier: {tier}")
```

```
Patient: {'gender': 'Male', 'age': 67, 'hypertension': 0, 'heart_dis
Predicted stroke probability: 0.757
Risk tier: HIGH
```

The patient (67- year old male) hit multiple of the most highly weighted features of this model at the same time, heart disease present, high average glucose, senior age band, which is a great way of ensuring the model is in step with medical judgment, and not a black box.

7. Discussion

Strengths

- This pipeline addresses the class imbalance two different ways (class weighting, SMOTE), and checks which is better using CV F1 instead of taking one of them as granted.
- It leverages a leak-free scikit-learn Pipeline, integrates annotations on two engineered features that rely on clinical thresholds (high-glucose flag, age bands) and provides comparisons of three model families ranging from an interpretable baseline to a boosted-tree model, for individual- prediction explanations using SHAP.

Limitations

- Only ~249 positive cases, in a dataset of 5,110, is a too small sample for a rare 'event' (such as a case); the facts in performance estimate (particularly recall) are unconfirmed with wide uncertainty and should be re-validated on a larger, prospective sample before use at deployment time.
- No timescale information (e.g. duration of hypertension) is available in the data set and so, there is limited ability for the model to learn the "risk trajectory".
- There is a large category of smoking status that is not smoking, which may be associated with other factors, such as age (likely, and changes in data collection may be a hidden variable), and may be a hidden confounding variable.
- SMOTE creates synthetic instances in the feature space; sometimes produces unnatural values (e.g., a child with a senior's risk profile), but causes them and goes against class weightings so be careful before believing it.

Business implications & recommendation

They tend to be very consistent factors: age, average glucose level, hypertension and heart disease which all seem to represent findings from clinical factors, which reassuringly agrees with current clinical risk factors.

Ideally, a tolerant deployment is not an accept/reject gate, but rather a "risk"-based tool: patients whose probabilities exceed some chosen cut-off (adjustable based on the PR curve, with the trade-off being that a higher cut-off means that a patient with a true positive will be missed, but a lower cut-off results in more false alarm flagged for clinician manual review, which is more costly than a true positive missed).

Explainability.

Explainability of the model at a patient level and at a global level: the Random Forest's feature importances plus the SHAP summary plot provide information on what were the factors that moved a patient's risk score higher.

Deployment

As a decision support signal in addition to clinical judgement — the threshold and feature importances should be periodically verified as new patient data is acquired and retrain regularly.

However, the model should not be considered as a standalone/ automated decision for care, as the number of instances of positive class is limited.